SOFTWARE-DEFINED NETWORKING AND OPENFLOW WITH MININET

Rafael Olim 93326

Diogo Ribeiro 102484

1. INTRODUCTION

This project explores the concept of Software-Defined Networking (SDN) and its practical application using the OpenFlow protocol and the Mininet emulator. SDN introduces a new paradigm in network management by decoupling the control plane from the data plane, enabling centralized, programmable, and more efficient network control compared to traditional architectures.

The lab is structured as a set of tasks designed to gradually move from basic concepts to more advanced implementations. Starting with simple hub-like behavior, it advances to learning switch logic, flow-based switching, and ultimately a custom Quality-of-Service (QoS) topology. Through this progression, the project highlights the flexibility, control, and traffic optimization benefits that SDN brings to modern networking.

2. OBJECTIVES

The main goals of this project are to:

- Understand the fundamental concepts of SDN and OpenFlow.
- Gain hands-on experience in configuring and programming an SDN controller.
- Implement different network behaviors: hub, learning switch, flow-based switch, and QoS-based routing.
- Assess the impact of these approaches on traffic management through packet capture analysis and control traffic evaluation.
- Develop a custom network topology to demonstrate how SDN can enforce routing policies based on link characteristics.

By the end of the lab, the following functionalities are expected:

1. **Learning Switch**: Packets are forwarded to specific ports after the MAC address of a host is learned, instead of flooding all ports as in hub behaviour.
2. **Flow-Based Network**: The controller installs flows in switches so that subsequent packets of the same flow to be handled locally, thereby reducing controller–switch traffic.
3. **Custom QoS Topology**: The SDN controller enforces a traffic distribution policy where flows are assigned to paths proportionally to their bandwidth (e.g., a 2:1 ratio between blue and red links).

2.1. Final Task Details

The objective of Task 3 is to extend the SDN implementation to a custom topology composed of six switches and six hosts, interconnected with links of different capacities (1000 Mb/s for blue links and 500 Mb/s for red links). The controller must install flow rules so that traffic distribution respects a 2:1 ratio, meaning that for every three active flows, two are routed through the higher-capacity blue links and one is routed through the lower-capacity red links.

The goal is to demonstrate how SDN can enforce Quality-of-Service (QoS) policies by dynamically balancing flows according to link characteristics, rather than relying on static, hardcoded routing. Validation will be carried out by generating multiple flows and confirming that the controller correctly allocates them across the available paths in the required proportion.

3. Experimental Methodologies

The experimental environment for all tasks is set up in **Mininet**, which provides a virtualized platform to emulate hosts, switches, and controllers within a single system.

3.1. Task 1 – Acting Like a Learning Switch

For this task, a single-switch topology with three hosts is deployed, where all hosts connect to the same switch, which in turn communicates with a remote SDN controller. This topology was created in Mininet using the following command: ***sudo mn --topo single,3 --mac --controller remote --switch ovsk***

We started by analyzing and understanding the functionality of the base code *of_tutorial.py* located at *pox/pox/misc/*. As expected, it behaved like a hub, forwarding every incoming packet on one port to all other ports on the switch.

After that the *of_tutorial.py* controller script was modified to implement learning switch functionality instead of simple hub behavior. The reasoning behind our approach was the following:

- Upon each **packet_in** sent from the switch (triggered by a flow table miss), the controller recorded the mapping between the packet's source MAC address and the corresponding ingress port.
- For subsequent packets with a known destination MAC, the controller instructed the switch, via an **ofp_packet_out** message, to forward them only to the correct port, rather than unnecessarily flooding all ports.

We then proceeded to validate the correctness of this implementation through the experiments detailed in Section 4.1.

3.2. Task 2 – Flow-Based Network (SDN Approach)

Task 2 employed the same topology as Task 1, consisting of a single switch connected to three hosts.

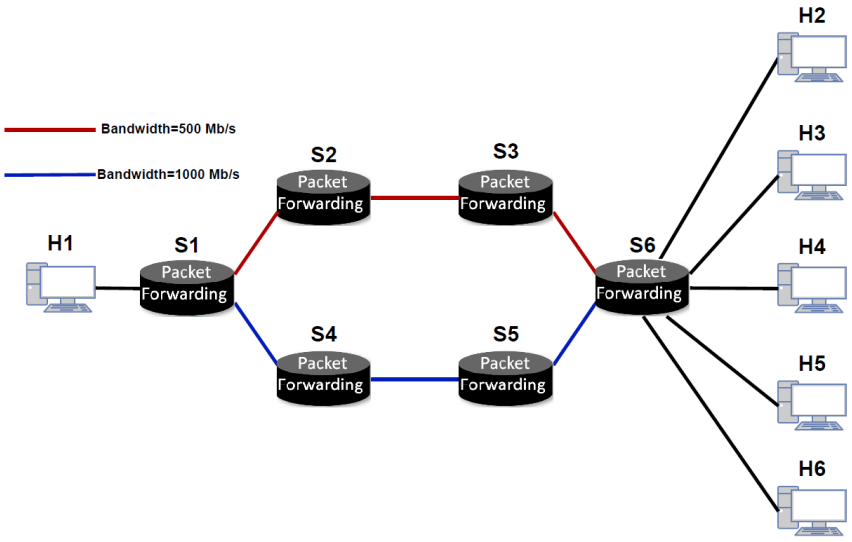
In this second task, the *of_tutorial.py* controller script was extended to install flow entries in the switch. When the first packet of a new flow arrived, it generated a **packet_in** message to the controller. As in Task 1, the controller learned the source MAC address and, if the destination MAC was already known, it responded with an **ofp_flow_mod** message. This flow entry matched the packet's fields (including the input port and MAC addresses) and specified the correct output port for all subsequent packets of the same flow. To prevent the flow table from growing indefinitely and exhausting memory, idle and hard timeouts were set. An additional improvement was handling buffered packets: if the packet was stored at the switch, the controller included the buffer_id in the flow_mod to forward it immediately, avoiding the overhead of resending the full packet to the switch.

This modification allowed the switch to handle subsequent packets locally without involving the controller, reducing controller-switch communication for established flows. Flooding still occurred if the destination MAC was unknown, ensuring proper delivery while new flows were learned.

To verify the implementation and assess its improvements, we performed functional tests and benchmarking as described in Section 4.2.

3.3. Task 3 – Custom Topology with Quality-Of-Service

For this task, a custom topology in Mininet consists of six hosts and six switches interconnected with heterogeneous link capacities: 1000 Mb/s for the blue links (s1, s2, s3, s6) and 500 Mb/s for the red links (s1, s4, s5, s6). The topology was implemented using a Python script (*lab1_topology.py* that was placed at *mininet/custom/*) that defined the hosts, switches, and their interconnections, as Mininet allows for flexible topology creation beyond the default single-switch configuration. This topology is represented below.



The controller logic was extended so that flows were distributed proportionally to the available link capacity, following a 2:1 ratio between the blue and red links. This meant that for every three active flows in the network, two were routed across the higher-capacity blue paths while one was routed across the lower-capacity red paths. To achieve this, the controller dynamically installed flow entries, avoiding hardcoded rules tied to specific host addresses, so that load balancing could adapt depending on the order and timing of traffic generation.

To address this problem, we divided the switches into two categories: **Core** and **Edge** switches. **Core** switches are those that do not connect directly to hosts but instead link switches together along the path. Since they only forward traffic between switches, their logic does not require major modifications compared to Task 2. In fact, they can essentially operate as learning switches that support flow installation. **Edge** switches, on the other hand, are more complex because they connect to both hosts and switches. Their position in the topology makes them susceptible to creating loops if packets are forwarded without additional control logic. The **SwitchHandler** class was introduced as the base class for both Core and Edge switch handlers, implementing the common functionality such as maintaining the connection to the switch and the MAC address-to-port mapping. This is similar to the **Tutorial** class from previous tasks but extended to handle multiple switches linked to each other.

For simplicity, we assumed a static topology with links hardcoded from the output of Mininet's ***net*** command, which meant that LLDP was not required. Using this information, we built per-switch maps (Python dictionaries) that efficiently identify the neighbouring switch for a given port or determine the port corresponding to a particular neighbour.

For the flooding logic, when the output port was unknown or the packet was broadcast, Edge switches required special handling due to the risk of forwarding loops. To surpass

this issue, upon **packet_in** the controller verifies if ingress port corresponds to another switch, in that case it instructs the switch to forward the packet only to non-switch ports. Otherwise, the switch forwards the packet not only to hosts but also selects a neighbour switch based on a weighted probability, it cannot select both since it will lead to a loop or waste of resources. This probability helps to maintain the 2:1 ratio, even for this type of events.

Flow installation also required careful handling to preserve the intended 2:1 traffic distribution. Since flows may have different lifetimes, it was necessary to track the number of active flows per path. This was implemented using a **cookie** (with a global counter) embedded in **ofp_flow_mod** messages. Each cookie was associated with a path, and the switch was instructed to notify the controller when the corresponding flow expired (due to timeouts). The ingress switch (an Edge switch) then decided which path to use whenever multiple options existed, marking MAC-to-port entries as -1 to indicate ambiguity. The final path selection was based on the current number of active flows in the high-bandwidth path and in the low-bandwidth path, approximating the 2:1 ratio dynamically.

The paths between switches 1 and 6 were hardcoded in the implementation to simplify the path exploration process, as the network topology is known a priori. This approach eliminates the need to implement a BFS algorithm that stops after finding all shortest paths. In this specific topology, the number of shortest paths is only two.

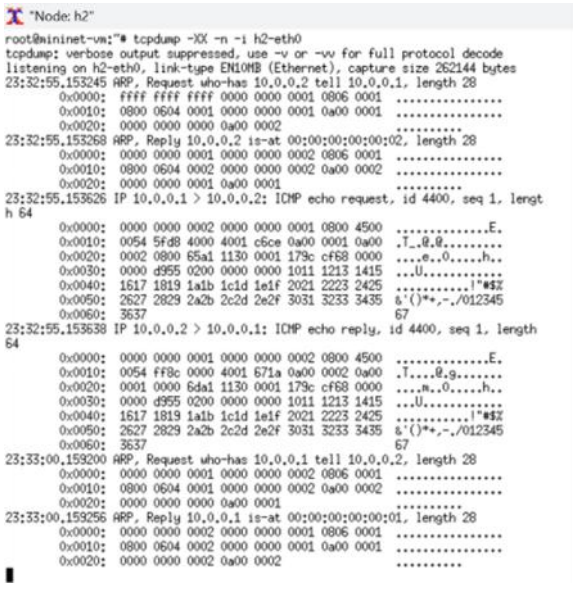
Once a path was selected, flow rules were installed end-to-end, starting from the egress switch and working backward toward the ingress switch. This approach minimized packet-in events by pre-installing rules along the path, saving three unnecessary controller interactions. We decided not to install the reverse path automatically, since some flows may be unidirectional. While this choice may result in an additional **packet_in** for reverse traffic, it avoids wasting flow table entries on unused paths. Whenever possible, we leveraged switch buffering to resend the original packet back through the ingress switch, reducing overhead further. Because the topology and switch-to-port associations were known in advance from the static mapping, port selection during flow installation could be performed efficiently in most cases.

Correctness and results described in Section 4.3.

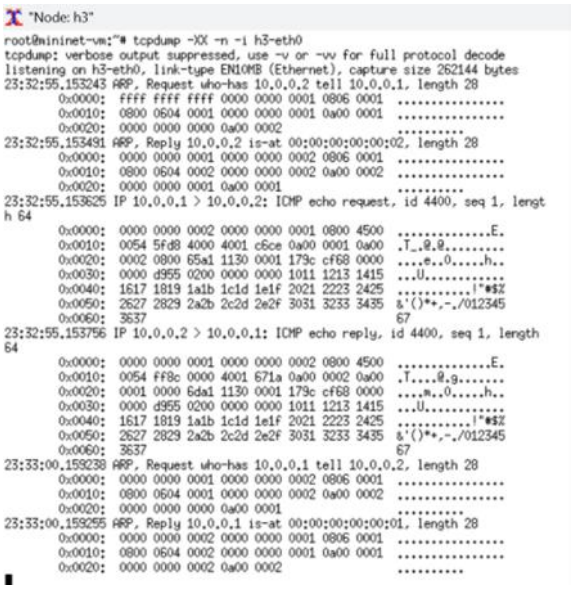
4. RESULT ANALYSIS

4.1. TASK 1 – ACTING LIKE A LEARNING SWITCH

In this task we started testing the forwarding hub controller by pinging h2 from h1 (using the command ***ping –c1 10.0.0.2*** from the host1) obtaining the following:



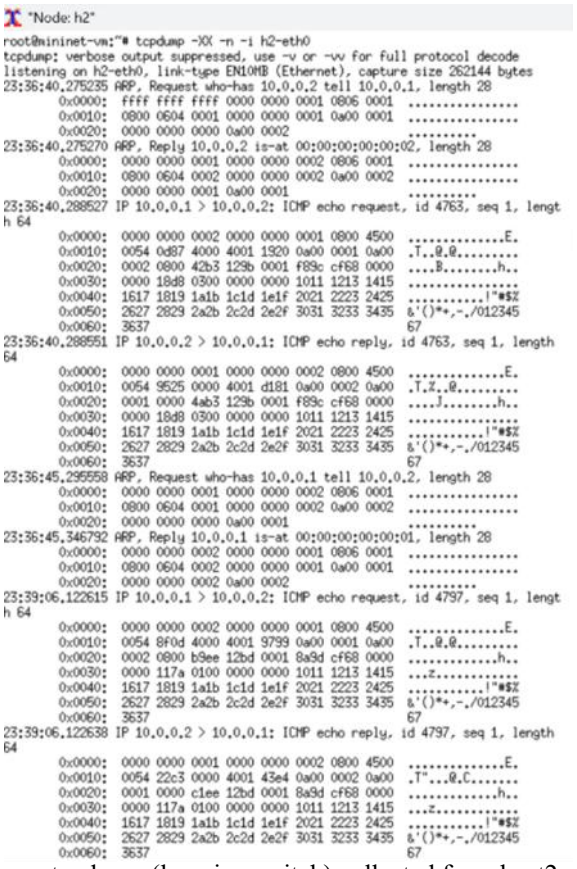
tcpdump (hub) collected from host2



tcpdump (hub) collected from host3

From the results we see every packet being forward to all hosts even though the destination MAC does not correspond to the host's MAC address.

After modifying the controller to implement learning switch functionality, we validated its behaviour using the same ping command, this time performing the ping twice. As observed, host3 captured only a single ARP request at the start of communication, since ARP is broadcast. Once the controller had learned the MAC-to-port mappings, no further ICMP or unicast ARP packets destined for h2 appeared at h3.



tcpdump (learning switch) collected from host2



tcpdump (learning switch) collected from host3

From this fact, we can confirm that, unlike in hub mode, the switch no longer flooded traffic to all hosts. Only the intended destination (h2) received the ICMP and ARP packets, while other hosts (h3) remained without traffic after the initial ARP. This demonstrated that the switch was correctly learning and using MAC address-to-port associations.

To benchmark the network, we tested TCP throughput between each host pair using the Mininet ***iperf*** command. Each connection was tested three times in both directions to ensure consistent measurements. The command ***pingall*** was used previously to populate the switch's MAC table and the hosts' ARP caches, ensuring that subsequent traffic is forwarded directly to the correct ports without unnecessary flooding.

We obtained the following results:

- Total Average **Hub** – source to destination 16.27 Mbits/sec and destination to source 18.64 Mbits/sec
- Total Average **Learning Switch** – source to destination 16.59 Mbits/sec and destination to source 18.88 Mbits/sec

As the results show, the throughput improvement from the hub to the learning switch is minimal given that the main overhead comes from switch-controller communication (has significantly lower bandwidth) caused by flow table misses, which affects performance more than the reduction in flooding.

4.2. TASK 2 – FLOW-BASED NETWORK (SDN APPROACH)

To validate the implementation, ***tcpdump*** and ***ping*** were first used to verify basic connectivity, and as expected, the traffic exchanged between hosts and the switch remained identical to Task 1.

For analyzing the OpenFlow control messages exchanged between the switch and the controller, **Wireshark** was used with the filter: ***of and not (of10.echo_request.type or of10.echo_reply.type).*** This allowed us to capture only the relevant OpenFlow packets, excluding echo requests and replies. The test scenario consisted of sending five ICMP packets from host1 to host2 using the command ***ping –c5 10.0.0.2***. This ensured that the flow contained more than one packet, making it possible to observe how the controller installed and reused flow entries. The captured results were as follows:

2468	82.444481000	127.0.0.1	127.0.0.1	OF 1.0	74	of_barrier_reply	52212	6633
3163	107.157927000	00:00:00_00:00:01	Broadcast	OF 1.0	126	of_packet_in	52212	6633
3164	107.170788000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3166	107.171158000	00:00:00_00:00:02	00:00:00_00:00:01	OF 1.0	126	of_packet_in	52212	6633
3167	107.171565000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3168	107.171807000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3169	107.172184000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3170	107.172828000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3171	107.173199000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3210	108.159949000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3212	108.208166000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3214	108.208562000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3216	108.210451000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3264	109.161725000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3266	109.200536000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3268	109.200913000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3270	109.202820000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3300	110.164000000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3301	110.188450000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3303	110.188855000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3304	110.189744000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3332	111.167022000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3333	111.179406000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3335	111.179840000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3336	111.181559000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3369	112.179098000	00:00:00_00:00:02	00:00:00_00:00:01	OF 1.0	126	of_packet_in	52212	6633
3370	112.218349000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3372	112.218730000	00:00:00_00:00:01	00:00:00_00:00:02	OF 1.0	126	of_packet_in	52212	6633
3373	112.220513000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3411	113.324009000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3412	113.359072000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3414	113.359474000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3415	113.361211000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3454	114.326418000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3455	114.349602000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3457	114.350324000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3458	114.352554000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3492	115.327626000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3493	115.333307000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3495	115.333694000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3496	115.341618000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3539	116.330097000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3541	116.378762000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3543	116.379149000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3545	116.381156000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3580	117.333392000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52212	6633
3582	117.376293000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212
3584	117.376705000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52212	6633
3586	117.378531000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52212

Wireshark OpenFlow packets capture learning switch

354	6.565325000	127.0.0.1	127.0.0.1	OF 1.0	74	of_barrier_reply	52196	6633
13109	127.512709000	00:00:00_00:00:01	Broadcast	OF 1.0	126	of_packet_in	52196	6633
13111	127.518809000	127.0.0.1	127.0.0.1	OF 1.0	90	of_packet_out	6633	52196
13113	127.519229000	00:00:00_00:00:02	00:00:00_00:00:01	OF 1.0	126	of_packet_in	52196	6633
13114	127.521089000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
13115	127.522062000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52196	6633
13116	127.532280000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
13117	127.533457000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52196	6633
13118	127.538070000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
14325	132.546150000	00:00:00_00:00:02	00:00:00_00:00:01	OF 1.0	126	of_packet_in	52196	6633
14327	132.554764000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
14329	132.555873000	00:00:00_00:00:01	00:00:00_00:00:02	OF 1.0	126	of_packet_in	52196	6633
14330	132.557454000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
25709	186.244615000	10.0.0.1	10.0.0.2	OF 1.0	182	of_packet_in	52196	6633
25721	186.275807000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
25723	186.276864000	10.0.0.2	10.0.0.1	OF 1.0	182	of_packet_in	52196	6633
25724	186.282668000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
26853	191.281279000	00:00:00_00:00:02	00:00:00_00:00:01	OF 1.0	126	of_packet_in	52196	6633
26900	191.309472000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196
26902	191.312001000	00:00:00_00:00:01	00:00:00_00:00:02	OF 1.0	126	of_packet_in	52196	6633
26903	191.314199000	127.0.0.1	127.0.0.1	OF 1.0	146	of_flow_add	6633	52196

Wireshark OpenFlow packets capture flow-based network

In the learning switch (Task 1), every ICMP packet triggered a **packet_in** message to the controller. Thus, when a ping with five ICMP packets was performed, five separate **packet_in** messages were observed in each direction, in addition to the ARP packets required for address resolution. This highlights the continuous controller-switch communication overhead, since the controller must process each packet individually, even when packets belong to the same flow.

In the flow-based switch (Task 2), only the first packet of each flow generated a **packet_in**. For the initial ping of 5 packets, the switch sent a single **packet_in** to the controller for each direction. The controller responded with an **ofp_flow_mod** message, installing a flow entry for that ICMP traffic. As a result, the remaining four ICMP packets were forwarded directly by the switch without additional controller interaction.

These captures confirmed a significant reduction in control traffic, demonstrates the efficiency of the flow-based approach, reducing overhead on the controller and improving scalability.

To benchmark the network, we proceeded as in Section 4.1. Obtaining the results:

- Total Average **Learning Switch** – source to destination 16.59 Mbits/sec and destination to source 18.88 Mbits/sec
- Total Average **Flow-Based Network** – source to destination 30.05 Gbits/sec and destination to source 30.13 Gbits/sec

These results demonstrate a throughput improvement from the order of **Mbits/sec** to **Gbits/sec** when moving from the learning switch to the flow-based network. The gain is explained by the elimination of continuous **packet_in** processing in the learning switch. By installing flow entries, the flow-based approach enables packets to be forwarded directly by the switch, achieving near line-rate performance closer to hardware capabilities.

4.3. TASK 3 – CUSTOM TOPOLOGY WITH QUALITY-OF-SERVICE

After placing *lab1_topology.py* in *mininet/custom/* we created the topology from the command: *sudo mn --custom ~/mininet/custom/lab1_topology.py --topo mytopo --controller=remote*

Placing the controller script in *pox/pox/misc/* as *lab1_controller.py* we were able to call it using: *./pox/pox.py log.level --DEBUG misc.lab1_controller*

To validate the results traffic is generated between different host pairs using Mininet tools such as **iperf** to establish multiple concurrent flows, at the same time **tcpdump** and **wireshark** are used to verify the paths chosen by each flow. By observing that the majority of flows go through the higher capacity link while some go through the lower capacity link the correct proportional distribution of the flow is confirmed.

The benchmark of the connectivity between the hosts connected to switch6 was identical to the results obtained for the flow-based network in Section 4.2 since these links were not limited in terms of bandwidth.

However, the situation is different when testing connectivity between h1 and the other hosts, as it is located on the opposite side of the topology. Since each flow uses only a single path at a time, the SDN controller distributes traffic according to the 2:1 ratio: 2/3 of the traffic traverses the 1000 Mbit/s links, while 1/3 goes through the 500 Mbit/s links. Therefore, the expected throughput is approximately 833 Mbit/s, which is still higher than the throughput actually observed:

- Total Average between host1 and the other hosts – source to destination 683.4 Mbits/sec and destination to source 701.9 Mbits/sec

This performance gap is explained by the control overhead from handling **packet_in** events and flow installations, imperfect flow distribution since the controller balances only by the number of flows rather than actual traffic demand, limitations of Mininet's software-based emulation, and effects of flow timeouts that may introduce additional control messages to reinstall flows. To achieve the full 1500 Mb/s bandwidth, it would be necessary to generate parallel traffic across both paths. This was not tested in the current host-to-host ***iperf*** measurements, which only used single flows per connection.

5. Conclusions

5.1. Task 1 – Acting Like a Learning Switch

This task successfully showed the improvement of a learning switch over hub behavior. By eliminating unnecessary flooding, the learning switch ensured more efficient use of network resources and avoided exposing irrelevant traffic to non-destination hosts. This implementation provided a practical foundation for the more advanced flow-based forwarding explored in subsequent tasks.

5.2. Task 2 – Flow-Based Network (SDN Approach)

Task 2 successfully demonstrated how SDN controllers can install flow entries in switches to offload repetitive packet handling. This reduces control plane overhead and ensures that the controller is only consulted when new flows are established, which is an advantage of the SDN paradigm.

5.3. Task 3 – Custom Topology with Quality-Of-Service

Task 3 successfully demonstrated how SDN can be used to implement Quality-of-Service policies in a custom topology with different link capacities. By programming the controller to install flows in a 2:1 ratio across high-capacity (1000 Mb/s) and low-capacity (500 Mb/s) links, the network was able to balance traffic more efficiently than with static or hardcoded routing.

The results confirmed that flows were distributed according to link bandwidth, with higher-capacity paths carrying proportionally more traffic. This highlighted the flexibility of SDN in enforcing dynamic traffic engineering policies, showing that routing decisions can be adapted to network characteristics rather than relying on the traditional shortest path forwarding.

Overall, this task illustrated the power of SDN for QoS-aware load balancing, ensuring that available resources are utilized efficiently and network performance is optimized in a programmable way.

6. APPENDIX

Modified code for task 3.1 (the symbol ***(...)*** represent missing parts of code that were unmodified from the original *of_tutorial.py*):

```
(...)

class Tutorial (object):

(...)

  def act_like_switch (self, packet, packet_in):
    """
    Implement switch-like behavior.
    """

    # Learn the port for the source MAC
    self.mac_to_port[packet.src] = packet_in.in_port

    if self.mac_to_port.get(packet.dst):
      # Send packet out the associated port
      self.resend_packet(packet_in, self.mac_to_port[packet.dst])

    else:
      # Flood the packet out everything but the input port
      self.resend_packet(packet_in, of.OFPP_ALL)


  def _handle_PacketIn (self, event):
    """
    Handles packet in messages from the switch.
    """

    packet = event.parsed # This is the parsed packet data.
    if not packet.parsed:
      log.warning("Ignoring incomplete packet")
      return

    packet_in = event.ofp # The actual ofp_packet_in message.
    self.act_like_switch(packet, packet_in)

(...)
```

Modified code for task 3.2 (the symbol ***(...)*** represent missing parts of code that were unmodified from the original *of_tutorial.py*):

```
(...)

class Tutorial (object):

(...)

  def act_like_switch (self, packet, packet_in):
    """
    Implement switch-like behavior.
    """

    # Learn the port for the source MAC
    self.mac_to_port[packet.src] = packet_in.in_port

    if self.mac_to_port.get(packet.dst):
      # Send packet out the associated port
      log.debug("Installing flow for %s.%i -> %s.%i" % (packet.src,
packet_in.in_port, packet.dst, self.mac_to_port[packet.dst]))

      msg = of.ofp_flow_mod()

      # Set fields to match received packet
      msg.match = of.ofp_match.from_packet(packet, packet_in.in_port)

      # Set other fields of flow_mod
      msg.idle_timeout = 10
      msg.hard_timeout = 30
      if packet_in.buffer_id and packet_in.buffer_id != -1:
        msg.buffer_id = packet_in.buffer_id
      else:
        msg.data = packet_in

      # Output action and send
      out_port = self.mac_to_port[packet.dst]
      action = of.ofp_action_output(port=out_port)
      msg.actions.append(action)
      self.connection.send(msg)

    else:
      # Flood the packet out everything but the input port
      self.resend_packet(packet_in, of.OFPP_ALL)
```

```
  def _handle_PacketIn (self, event):
    """
    Handles packet in messages from the switch.
    """

    packet = event.parsed # This is the parsed packet data.
    if not packet.parsed:
      log.warning("Ignoring incomplete packet")
      return

    packet_in = event.ofp # The actual ofp_packet_in message.
    self.act_like_switch(packet, packet_in)

(...)
```

Topology and controller scripts for the final task (task 3) are present in G02_Lab1.zip as *lab1_topology.py* and *lab1_controller.py* respectively.

The complete code for task 1 and task 2 is also present in the zip as *of_tutorial_learning_switch.py* and *of_tutorial_sdn.py* respectively.